

Yes. Your usual ML workflow is a good base, but this project needs one important change:

You are not doing a normal supervised classification project.

Your main goal is anomaly detection using an Autoencoder, where the model learns what normal network traffic looks like and flags flows that reconstruct badly.

Your `cicids2017_binary_balanced.csv` is actually well suited because the columns you showed are network-flow features: packet counts, packet lengths, flow duration, IATs, flags, bytes/s, packets/s, etc.

📁 AI Cybersecurity — Network Anomaly Detection Project

📁 Project objective

Goal: Detect abnormal/malicious network flows using an Autoencoder.

Basic idea:

```
Network Flow
  ↓
Preprocessing
  ↓
Normal traffic → Train Autoencoder
  ↓
Encoder
  ↓
Latent Representation
  ↓
Decoder
  ↓
Reconstructed Flow
  ↓
Reconstruction Error
  ↓
Low error → Normal 🟢
High error → Anomaly / Attack 🔴
```

Your `Attack_Binary` column is useful for evaluation, but the Autoencoder should ideally learn primarily from normal traffic (`Attack_Binary = 0`).

1. Importation des bibliothèques

You'll need roughly:

```
pandas
numpy
matplotlib
seaborn
sklearn
tensorflow / keras
streamlit
```

You don't need to decide the exact code yet. First build the project in stages.

2. EDA — Data Exploration

This part stays similar to your normal ML projects, but your questions change because this is cybersecurity/network data.

2.1 Analyse de forme

Identify target

Your dataset has:

Attack_Binary

Meaning:

0 → BENIGN / Normal
1 → ATTACK / Anomaly

Important: Don't immediately treat this as your model's target like you did with wine quality.

For Autoencoder anomaly detection:

Attack_Binary = 0
↓
Normal traffic
↓
Used to train Autoencoder

And:

Attack_Binary = 1
↓
Attack traffic
↓
Used mainly for testing/evaluation

So you can write in your notes:

Attack_Binary is the label used to distinguish normal traffic from attacks. For anomaly detection, the Autoencoder learns mainly from normal traffic and the label is used afterward to evaluate whether attacks receive higher reconstruction errors.

2.2 Number of rows and columns

You have:

Rows = 851,388
Columns = 53

Therefore:

52 network-flow features
+
1 label → Attack_Binary

That's much larger than your wine dataset, so don't make every visualization on the entire dataset.

2.3 Types of variables

You have:

```
33 int64
20 float64
```

And essentially all your predictors are numerical network-flow features.

No categorical encoding is required.

2.4 Missing values

Check:

```
NaN
Inf
-Infinity
```

This is particularly important for this dataset because network-flow datasets can contain problematic values in features such as:

```
Flow Bytes/s
Flow Packets/s
```

So your EDA should check both:

```
NaN → missing values
Inf → infinite values
```

3. Understand the Network Features

This is a new part I'd add to your normal workflow.

Before modeling, understand what your columns actually represent.

Example

Feature	Meaning
Destination Port	Destination port of the flow
Flow Duration	Duration of communication
Total Fwd Packets	Number of packets sent forward
Total Length of Fwd Packets	Total forward packet size
Fwd Packet Length Mean	Average forward packet size
Bwd Packet Length Mean	Average backward packet size
Flow Bytes/s	Bytes transmitted per second
Flow Packets/s	Packets transmitted per second

Flow IAT Mean	Average time between packets	
FIN Flag Count	Number of FIN flags	
ACK Flag Count	Number of ACK flags	
Average Packet Size	Average packet size	
Active Mean	Average active period	
Idle Mean	Average idle period	

Think:

A network flow is basically a summary of a communication session.

Instead of looking at every individual packet, CICIDS2017 gives you statistical characteristics of the flow.

4. Target / Class Distribution

Visualize:

Attack_Binary

For example:

0 → BENIGN
1 → ATTACK

Use a count plot/bar chart.

Your dataset is called:

cicids2017_binary_balanced.csv

so don't assume the distribution without checking it.

Write the actual result after:

value_counts()

5. Analyze Variables Separately

For numerical features, examine:

Distribution

Histogram
KDE
Boxplot

But 52 features is a lot.

You don't need 52 giant graphs.

You can select important representative features such as:

Flow Duration
Total Fwd Packets
Total Backward Packets
Flow Bytes/s
Flow Packets/s
Packet Length Mean
Packet Length Std
Flow IAT Mean
Average Packet Size

This helps you understand the network traffic.

6. Analyze Relationships Between Features

This part is different from your wine project.

You can use:

Correlation matrix

Feature ↔ Feature

This can reveal highly correlated/redundant features.

For example:

Total Fwd Packets
↓
Subflow Fwd Bytes

or:

Packet Length Mean
↓
Average Packet Size

You should investigate highly correlated features before blindly keeping everything.

7. Outliers

Here you need to change your thinking.

In a normal ML project you might say:

"We have outliers, let's remove them."

For cybersecurity:

An extreme value can actually be the interesting part.

For example:

Very high Flow Packets/s

Very unusual packet size
Very unusual Flow Duration

could represent abnormal network behavior.

Therefore:

Don't automatically remove outliers.

Instead, investigate them.

For anomaly detection, unusual observations are potentially valuable.

8. Data Cleaning

Now your preprocessing begins.

Check:

Missing values
Infinite values
Duplicates
Invalid values
Constant columns
Highly redundant columns

Especially:

NaN
+∞
-∞

9. Remove unnecessary columns

You don't have to randomly delete columns like you did with the wine dataset.

For this project, start with:

X = all network-flow features
y = Attack_Binary

Then investigate redundancy.

You can remove:

1. The label

From X:

Attack_Binary

because it must not be given to the Autoencoder as a feature.

2. Potential identifiers

If your dataset had things like:

Flow ID
Source IP
Destination IP
Timestamp

you would investigate whether they should be removed.

Your displayed dataset doesn't appear to contain these, so don't invent columns to remove.

10. Feature Engineering

This is optional.

Don't overcomplicate your first version.

Your dataset already contains many engineered network-flow features.

For example:

Flow Bytes/s
Flow Packets/s
Fwd Packets/s
Bwd Packets/s
Packet Length Mean
IAT statistics

These are already useful representations of network behavior.

So your first project can use the existing features.

11. Features Scaling ☐ VERY IMPORTANT

This is mandatory for your Autoencoder.

Why?

Imagine:

Flow Duration = 1000000
FIN Flag Count = 1
Packet Length Mean = 500

The neural network would see very different numerical scales.

Use:

StandardScaler

Conceptually:

```
Original data
  ↓
StandardScaler
  ↓
mean ≈ 0
std ≈ 1
  ↓
Autoencoder
```

12. Train/Test Split — BUT differently

This is one of the most important differences from your normal ML project.

You have:

```
Normal = 0
Attack = 1
```

For anomaly detection:

Training

Use normal traffic only:

```
Attack_Binary == 0
```

Then:

```
Normal training data
  ↓
Autoencoder learns normal patterns
```

Testing

Use:

```
Normal + Attack
```

So the Autoencoder can be tested on both.

Schema:

```
DATASET
├── Normal traffic
│   └── Train Autoencoder
└── Attack traffic
    └── Test Autoencoder
```

This is much more appropriate for anomaly detection than simply doing:

```
train_test_split(X, y)
```

and training on both classes.

13. Autoencoder Architecture

Now comes the core of your project.

Your model:

```
Input
↓
Dense
↓
Dense
↓
Latent Space
↓
Dense
↓
Dense
↓
Output
```

For example:

```
52 features
↓
32 neurons
↓
16 neurons
↓
8 neurons
↓
16 neurons
↓
32 neurons
↓
52 features
```

The important idea:

Encoder

52 → 32 → 16 → 8

compresses the network flow.

Latent space

8

contains a compact representation of normal network behavior.

Decoder

8 → 16 → 32 → 52

tries to reconstruct the original flow.

14. Train the Autoencoder

Training data:

Normal X_{train}

And:

Input = X_{train}
Target = X_{train}

Because the Autoencoder learns:

"How can I reconstruct normal network traffic?"

So:

Normal flow
↓
Autoencoder
↓
Good reconstruction

15. Reconstruction Error ☐☐☐

This is the heart of the project.

After reconstruction:

Original X
↓
Autoencoder
↓
Reconstructed X

Calculate:

Reconstruction Error
=
difference between original and reconstructed data

Usually:

MSE = Mean Squared Error

Conceptually:

Original:
[0.2, 0.5, 0.1, ...]

Reconstructed:
[0.21, 0.49, 0.11, ...]

↓

Small error

↓

Probably normal

But:

Original:
[0.2, 0.5, 0.1, ...]

Reconstructed:
[1.8, 2.4, 0.9, ...]

↓

Large error

↓

Potential anomaly ☐

Why?

Because the Autoencoder learned normal patterns.

It hasn't learned how to reconstruct unusual attack patterns very well.

16. Choose the Anomaly Threshold

You need a threshold:

reconstruction error > threshold

↓

ANOMALY

Otherwise:

reconstruction error <= threshold

↓

NORMAL

A simple approach:

Calculate errors on normal validation data

Then choose a threshold such as:

95th percentile

99th percentile

Example:

Normal reconstruction errors:

0.01

0.02

0.015
0.03
...

Suppose:

threshold = 0.05

Then:

Error = 0.02 → NORMAL ☐
Error = 0.03 → NORMAL ☐
Error = 0.08 → ANOMALY ☐

You can later compare different thresholds.

17. Evaluate the Anomaly Detector

Now finally use Attack_Binary.

Compare:

Actual:
0 = Normal
1 = Attack

with:

Predicted:
0 = Normal
1 = Anomaly

Then use:

Confusion Matrix

	Predicted	
	Normal	Attack
Actual Normal	TN	FP
Actual Attack	FN	TP

You already learned these in your evaluation course.

18. Metrics

Use:

Accuracy

Not the main metric.

Precision

Of everything detected as attack, how much was actually attack?

Recall □

Of all actual attacks, how many did we detect?

For cybersecurity, recall is especially important.

F1-score

Balance between:

Precision + Recall

ROC-AUC

Useful for evaluating the detector across thresholds.

You can also consider:

PR-AUC

especially if the real-world attack class is highly imbalanced.

19. Important Cybersecurity Interpretation

Don't just write:

"The model achieved 95% accuracy."

Explain what it means.

For example:

High Recall

↓

We detect many attacks

↓

Good for security monitoring

Low Recall

↓

Many attacks remain undetected

↓

Dangerous

But:

High Recall + very low Precision

↓

Many false alarms

↓

Security analyst gets overwhelmed

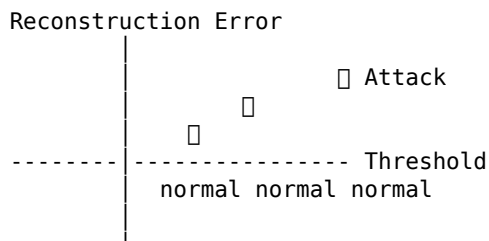
So you have a real cybersecurity tradeoff:

Detect more attacks
↓
Reduce false alarms

20. Visualize Reconstruction Error

This is an excellent visualization for your project.

Plot:



Ideally you should see:

Normal → lower errors
Attack → higher errors

If there is significant overlap, that's also an important result to discuss.

21. Compare Normal vs Attack

You can also visualize:

Distribution of reconstruction errors

for:

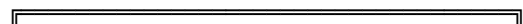
BENIGN
ATTACK

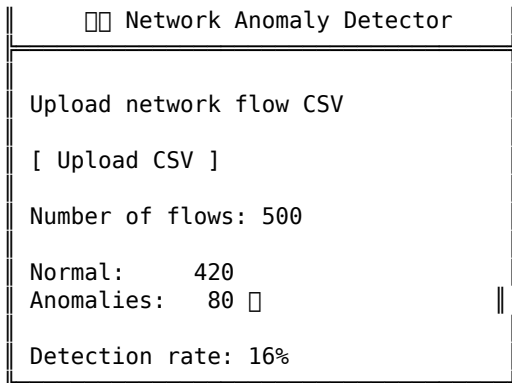
This is one of the most useful plots in the whole project.

22. Deployment — Streamlit

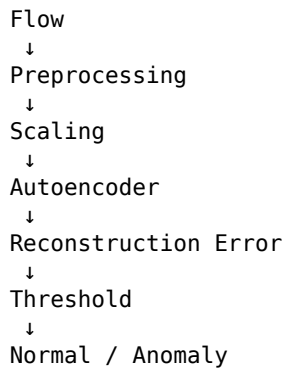
Then turn the project into a simple cybersecurity dashboard.

Something like:



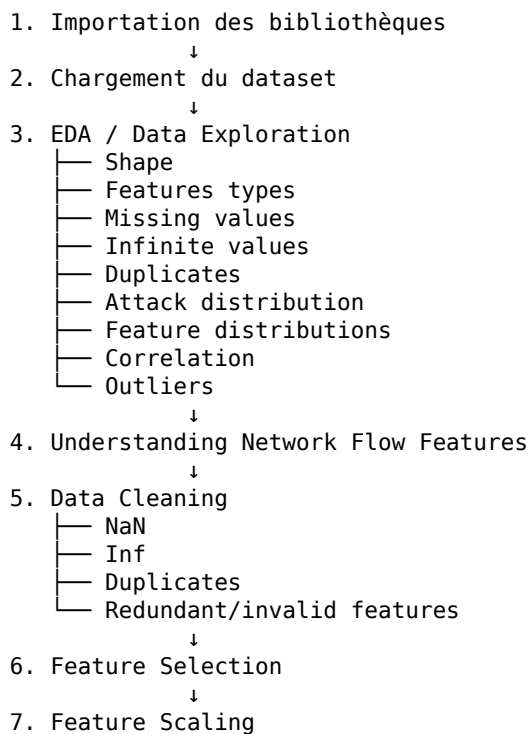


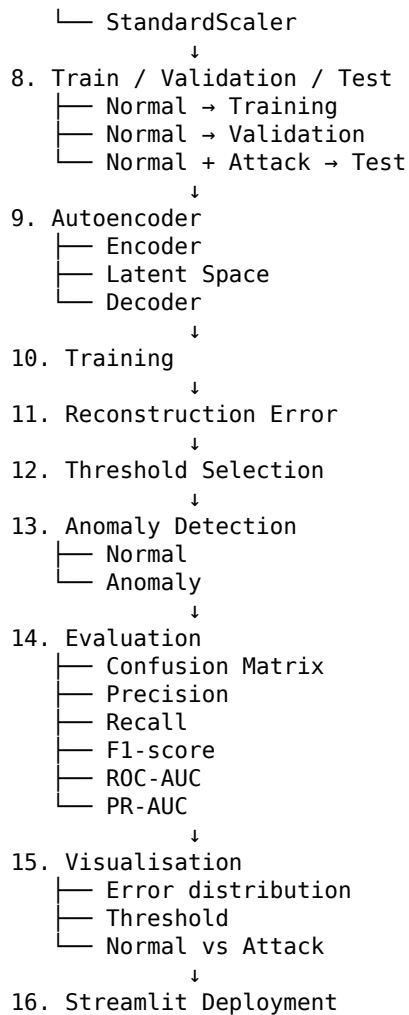
For each flow:



□ Your FINAL PROJECT PIPELINE

I would organize your project exactly like this:



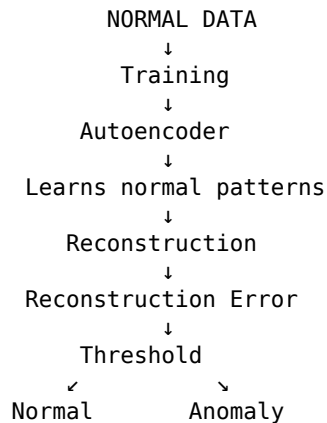


□ The biggest difference from your usual ML project

Your old project was basically:

Features → Target → Supervised Model → Prediction

Your new project is:



And only after that:

Predicted anomaly

↓
Compare with Attack_Binary
↓
Evaluate model

That's the conceptual point I would emphasize in your report/presentation: Attack_Binary is mainly your ground-truth label for evaluation, while the Autoencoder itself learns normal network behavior without needing attack labels during training.

One thing I would not do yet is start deleting 10-15 columns because they "look unimportant." With 52 flow features, first inspect missing/ ∞ values, distributions, correlations, and feature redundancy; then make the selection based on evidence.